

Online shoppers intention

Made by:

- Irvan Ilahibaks - 8081847
- Timon Baris - 2173182
- Waling van Dongen - 1176226

TA: Sita Newer

Vak: Data Analytics

30-9-2024

Task 1.1: Get dataset on screen

- To get a quick overview the functions .info and .describe are useful.
- It retrieves the amount of rows, data types, means, etc.
- Loc function also useful for specific searches

```
1 #2.0 Preprocessing
2
3 import pandas as pd
4 osiData = pd.read_csv('online_shoppers_intention.csv') #Initialize data
5 osiData.info() #viewing Data Types
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12330 entries, 0 to 12329
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Administrative         12330 non-null  int64
1   Administrative_Duration 12330 non-null  float64
2   Informational           12330 non-null  int64
3   Informational_Duration  12330 non-null  float64
4   ProductRelated         12330 non-null  int64
5   ProductRelated_Duration 12330 non-null  float64
6   BounceRates            12330 non-null  float64
7   ExitRates              12330 non-null  float64
8   PageValues             12330 non-null  float64
9   SpecialDay             12330 non-null  float64
10  Month                  12330 non-null  object
11  OperatingSystems       12330 non-null  int64
12  Browser                12330 non-null  int64
13  Region                 12330 non-null  int64
14  TrafficType            12330 non-null  int64
15  VisitorType            12330 non-null  object
16  Weekend                12330 non-null  bool
17  Revenue                12330 non-null  bool
dtypes: bool(2), float64(7), int64(7), object(2)
memory usage: 1.5+ MB
```

```
1 #2.0 Preprocessing
2
3 osiData.describe() #summary statistics
```

	Administrative	Administrative_Duration	Informational	Informational_Duration	ProductRelated	ProductRelated_Duration	BounceRates	ExitRates	PageValues
count	12330.000000	12330.000000	12330.000000	12330.000000	12330.000000	12330.000000	12330.000000	12330.000000	12330.000000
mean	2.315166	80.818611	0.503569	34.472398	31.731468	1194.746220	0.022191	0.043073	119.772389
std	3.321784	176.779107	1.270156	140.749294	44.475503	1913.669288	0.048488	0.048597	119.772389
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000	0.000000	7.000000	184.137500	0.000000	0.014286	11.977239
50%	1.000000	7.500000	0.000000	0.000000	18.000000	598.936905	0.003112	0.025156	11.977239
75%	4.000000	93.256250	0.000000	0.000000	38.000000	1464.157214	0.016813	0.050000	11.977239
max	27.000000	3398.750000	24.000000	2549.375000	705.000000	63973.522230	0.200000	0.200000	36.000000

Task 1.1: Get dataset on screen

```
osiData = pd.read_csv('online_shoppers_intention.csv')
```

```
maanden_order = ['Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Okt', 'Nov', 'Dec']
```

```
filtered_data = osiData.loc[  
    (osiData['Month'].isin(maanden_order)) &  
    (osiData['Revenue'] > 0)  
]
```

Monthly Revenue



```
month_revenue_data = filtered_data[['Month', 'Revenue']].copy()
```

```
month_revenue_data['Month_Order'] = month_revenue_data['Month'].apply(lambda x: maanden_order.index(x))
```

```
month_revenue_data = month_revenue_data.sort_values(by='Month_Order')
```

```
month_revenue_data = month_revenue_data.drop(columns=['Month_Order'])
```

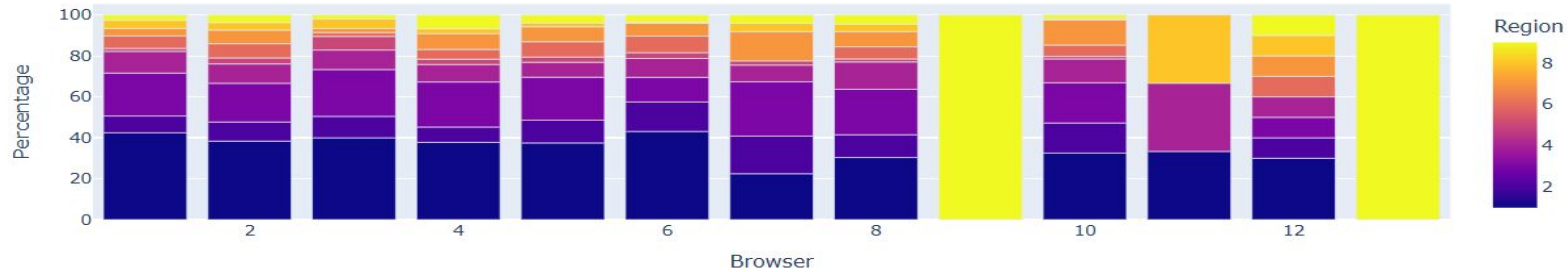
```
print(month_revenue_data[['Month', 'Revenue']].groupby('Month').sum())
```

```
print(month_revenue_data.to_string(index=False))
```

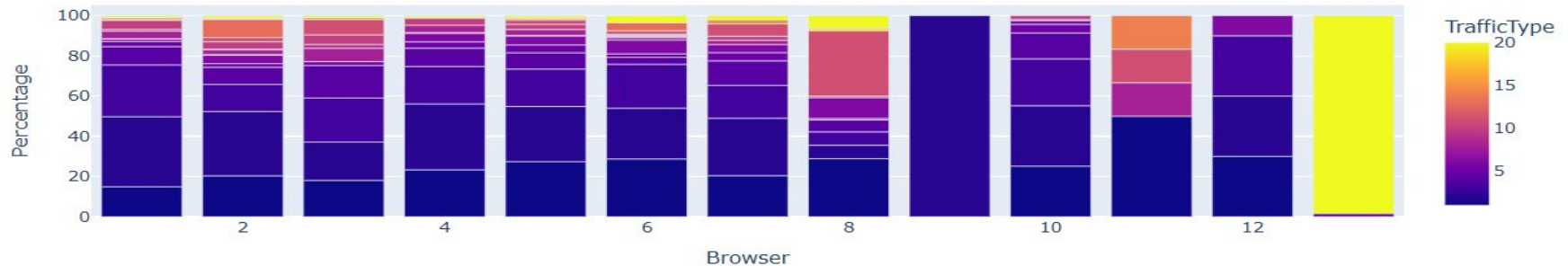


Task 1.2: Get dataset on screen

Percentage Distribution of Region by Browser

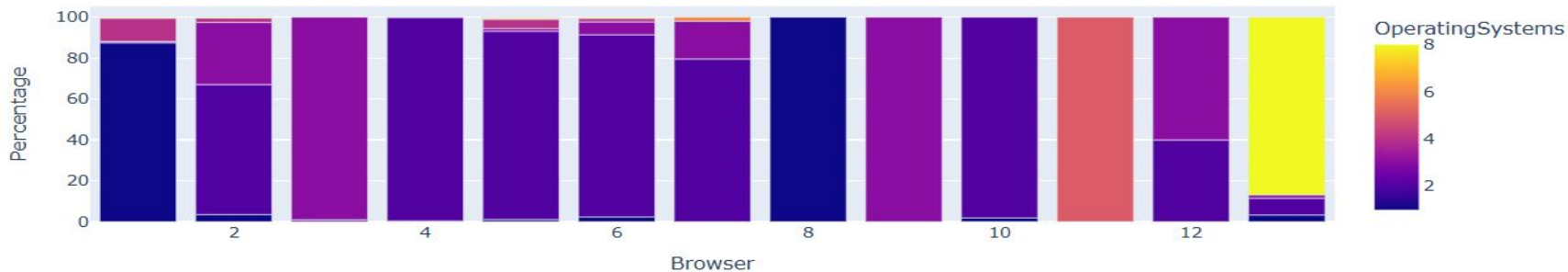


Percentage Distribution of Traffic Type by Browser



Task 1.2: Get dataset on screen

Percentage Distribution of Operating Systems by Browser



Percentage Distribution of Visitor Type by Browser



Task 2:

Preprocessing

```
# Check for missing values  
print(osiData.isnull().sum())
```

```
# Optionally fill missing values  
osiData.fillna(0, inplace=True)
```

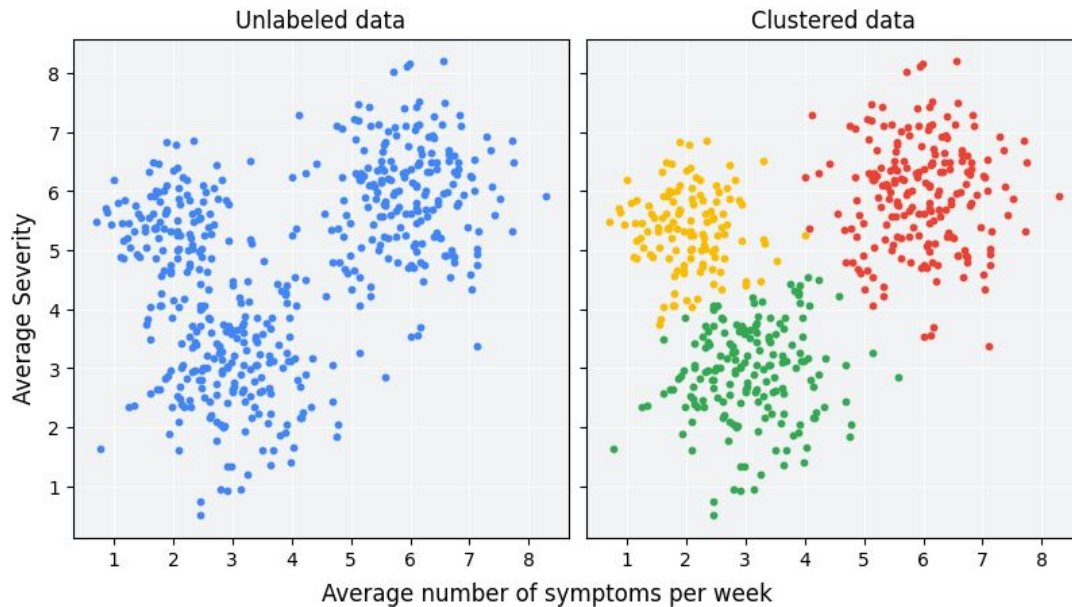
```
# Convert Weekend column from boolean to int (True=1, False=0)  
osiData['Weekend'] = osiData['Weekend'].astype(int)
```

```
numeric_features = osiData.select_dtypes(include=['int64', 'float64'])  
  
scaler = StandardScaler()  
scaled_features = scaler.fit_transform(numeric_features)  
  
scaled_data = pd.DataFrame(scaled_features, columns=numeric_features.columns)
```

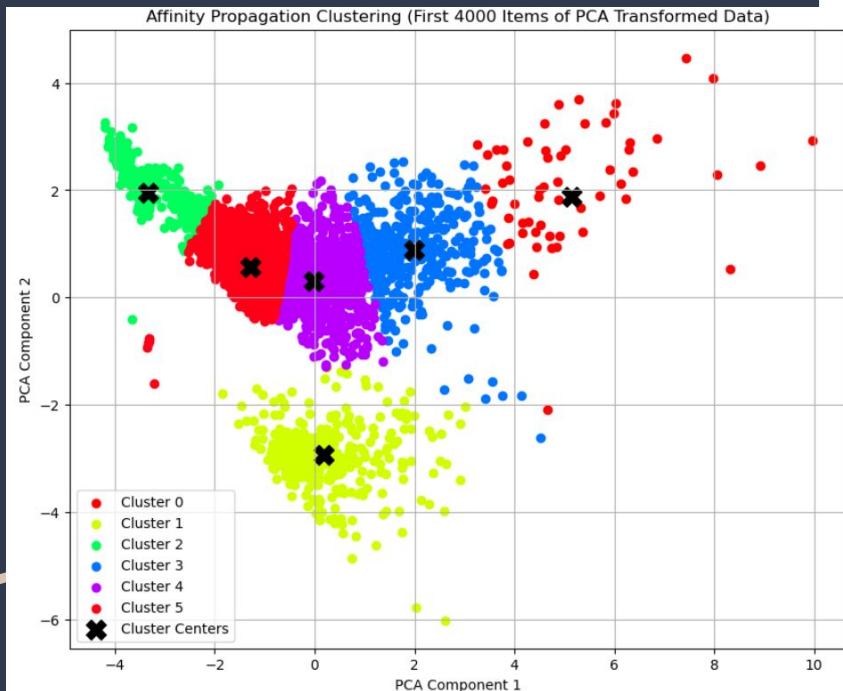
- Importance of preprocessing
- Raw Data Challenges
- Impact on Model Performance

Clustering

- Grouping similar data points
- Close to each other
- Well-separated
- Easy to interpret and meaningful



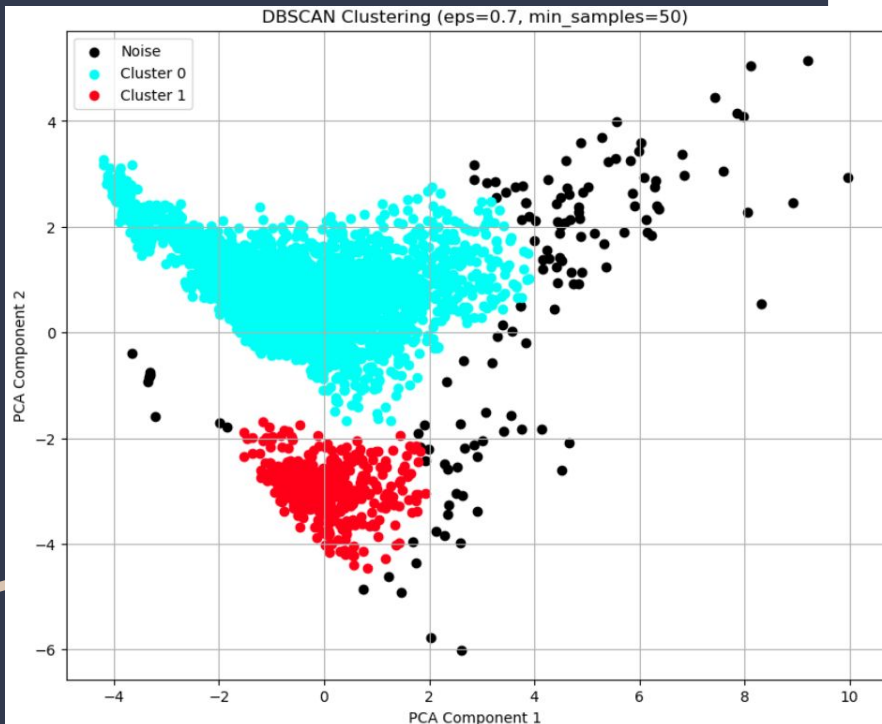
Task 3.1: Clustering algorithms



Affinity Propagation clustering

- Clusters based on message passing
- Scales well with large datasets
- Less effective with noisy data
- Sensitive to parameter settings

Task 3.2: Clustering algorithms



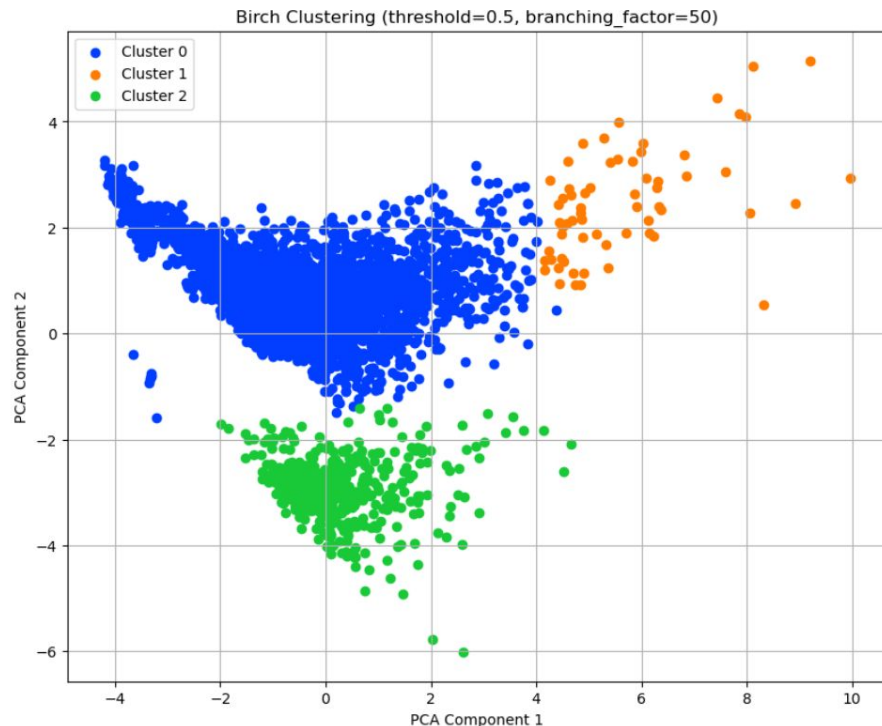
DBSCAN clustering

- Clusters based on density
- Efficient with large datasets
- Handles noise
- Less effective with varying densities

Task 3.3: Clustering algorithms

Birch clustering

- Clusters incrementally
- Still efficient despite scale
- Less efficient with complex data



Task 4.1: Evaluation of clustering methods

```
#usage with Affinity Propagation, DBSCAN, and BIRCH clustering
silhouetteAffinity = silhouetteScore(affDataClus, affinityLabels)
silhouetteDBSCAN = silhouetteScore(dbsClusData, dbscanLabels)
silhouetteBIRCH = silhouetteScore(birchClusData, birchLabels)

#print the silhouette scores for each clustering method
print(f'Affinity Propagation Silhouette Score: {silhouetteAffinity}')
print(f'DBSCAN Silhouette Score: {silhouetteDBSCAN}')
print(f'BIRCH Silhouette Score: {silhouetteBIRCH}')
```

```
Affinity Propagation Silhouette Score: 0.43482370396163544
DBSCAN Silhouette Score: 0.5328515643600106
BIRCH Silhouette Score: 0.5519469456834416
```

Silhouette score

- Similarity data point own cluster vs other clusters
- Average distance to other points in the same cluster
- Average distance to the nearest different cluster.
- ranges from -1 to +1
- closer to +1 → well separated cluster

$$s = \frac{b - a}{\max(a, b)}$$

Task 4.2:

Evaluation of clustering methods

Davies Bouldin Score

- Similarity data point own cluster vs other clusters
- Average distance within each cluster (compactness)
- Average distance between cluster centroids (separation)
- ranges from 0 to ∞
- Below 1 $\rightarrow$ well separated cluster

```
1 from sklearn.metrics import davies_bouldin_score
2 db_score_apf = davies_bouldin_score(AffDataClus, affinity_labels )
3 db_score_dbs = davies_bouldin_score(DbsClusData, dbscan_labels)
4 birch_db_score = davies_bouldin_score(BirchClusData, birch_labels)
5 print(f'DB-score for affinity: {db_score_apf}')
6 print(f'DB-score for DBScan: {db_score_dbs}')
7 print(f'DB-score for BIRCH: {birch_db_score}')
```

DB-score for affinity: 0.6192791052645137

DB-score for DBScan: 0.8893500214194164

DB-score for BIRCH: 0.5267612332603656

Task 4.3:

Evaluation of clustering methods

Calinski Harabasz Index

- Similarity data point own cluster vs other clusters
- Within-cluster variance (how tightly points are grouped)
- Between-cluster variance (how far apart clusters are)
- ranges from 0 to ∞
- Above 50 $\rightarrow$ well separated cluster

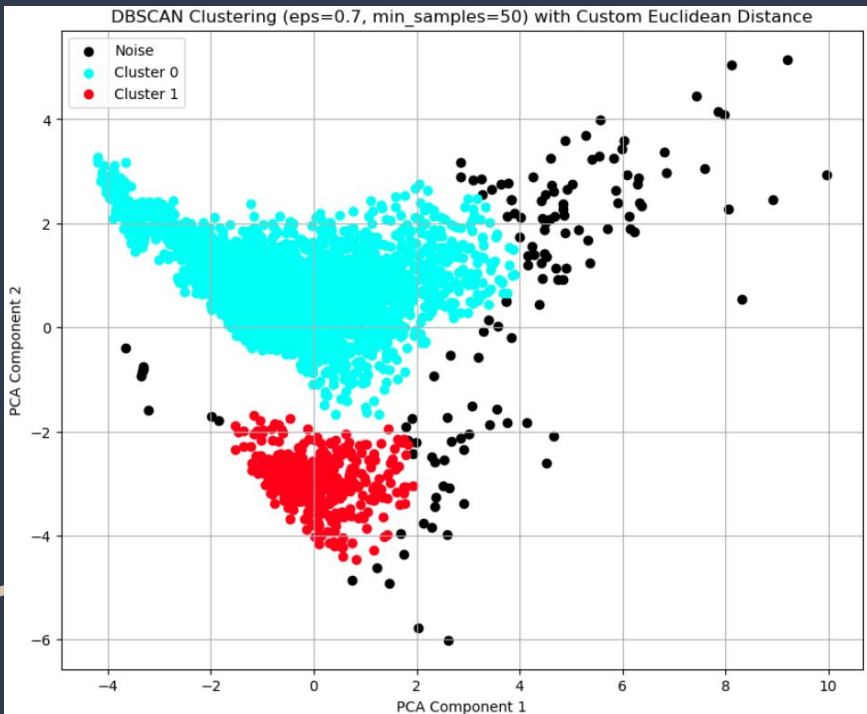
```
from sklearn.metrics import davies_bouldin_score

#calculate Davies-Bouldin scores for Affinity Propagation, DBSCAN and BIRCH clustering methods
dbScoreAffinity = davies_bouldin_score(affDataClus, affinityLabels)
dbScoreDBSCAN = davies_bouldin_score(dbsClusData, dbscanLabels)
dbScoreBIRCH = davies_bouldin_score(birchClusData, birchLabels)

#print the Davies-Bouldin scores
print(f'Davies-Bouldin Score for Affinity Propagation: {dbScoreAffinity}')
print(f'Davies-Bouldin Score for DBSCAN: {dbScoreDBSCAN}')
print(f'Davies-Bouldin Score for BIRCH: {dbScoreBIRCH}')
```

```
Davies-Bouldin Score for Affinity Propagation: 0.7074587118843737
Davies-Bouldin Score for DBSCAN: 0.8848354869752431
Davies-Bouldin Score for BIRCH: 0.6039077791014805
```

Task 5.1: Distance functions



Euclidean distance function

#5.1 Distance Functions Manually implant euclidian

```
def manualEuclideanDistance(p1, p2):  
    return sum((x1 - x2) ** 2 for x1, x2 in zip(p1, p2)) ** 0.5
```

```
dbscanData = osiPCA[:5000, :2]
```

```
eps = 0.7  
minSamples = 50
```

```
dbscan = DBSCAN(  
    eps=eps,  
    min_samples=minSamples,  
    metric=manualEuclideanDistance  
)
```

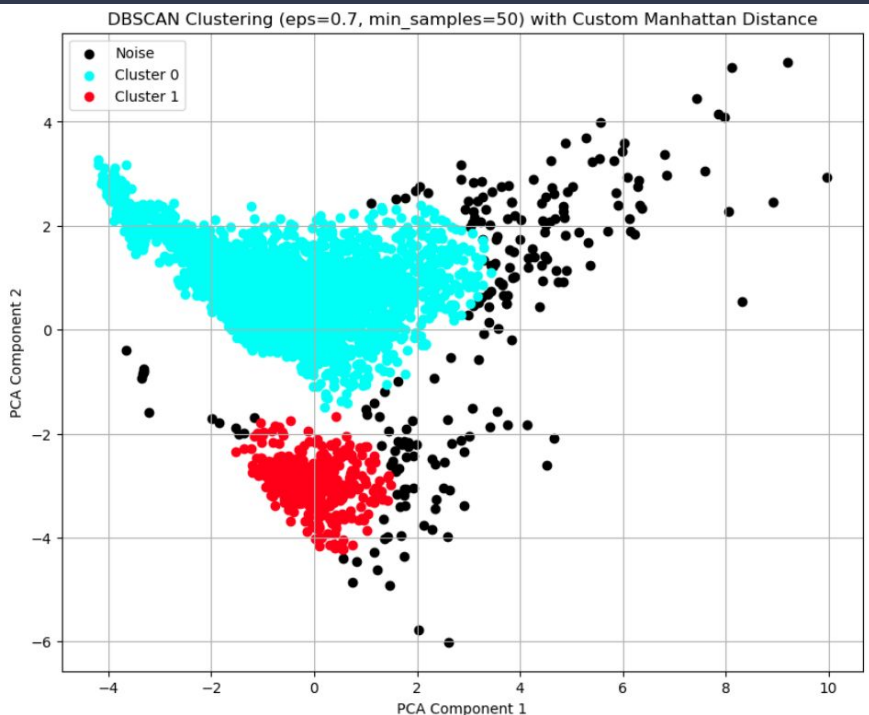
```
labelsEuclidean = dbscan.fit_predict(dbscanData)
```

```
uniqueLabels = np.unique(labelsEuclidean)  
colors = plt.colormaps['hsv'](np.linspace(0, 1, len(uniqueLabels)))
```

```
plt.figure(figsize=(10, 8))  
for i, label in enumerate(uniqueLabels):  
    clusterPoints = dbscanData[labelsEuclidean == label]  
    if label == -1:  
        plt.scatter(clusterPoints[:, 0], clusterPoints[:, 1], color='black', label='Noise')  
    else:  
        plt.scatter(clusterPoints[:, 0], clusterPoints[:, 1], color=colors[i], label=f'Cluster {label}')
```

```
plt.title(f'DBSCAN Clustering (eps={eps}, min_samples={minSamples}) with Custom Euclidean Distance')  
plt.xlabel('PCA Component 1')  
plt.ylabel('PCA Component 2')  
plt.legend()  
plt.grid()  
plt.show()
```

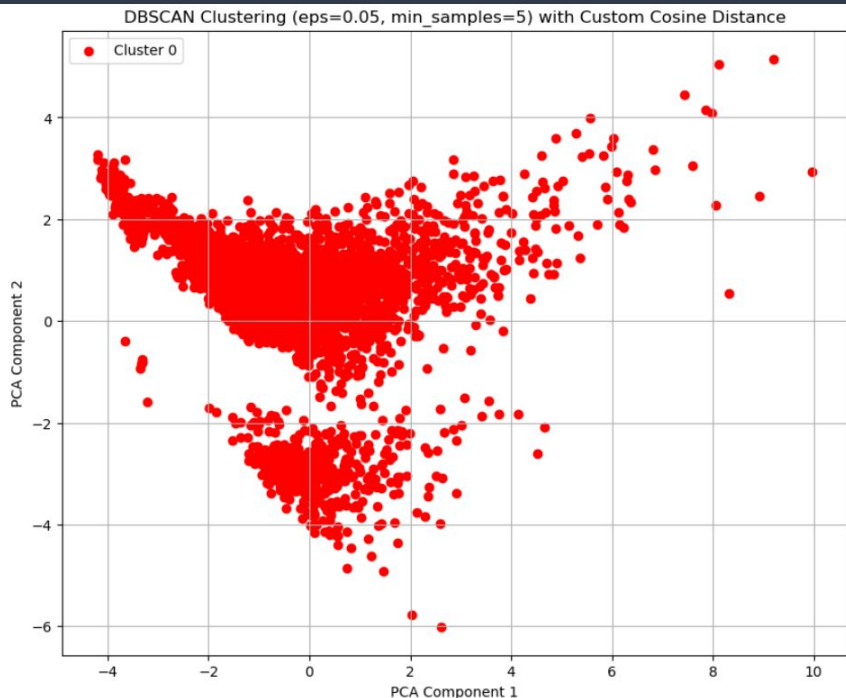
Task 5.2: Distance functions



Manhattan distance function

```
def manualManhattanDistance(p1, p2):  
    return sum(abs(x1 - x2) for x1, x2 in zip(p1, p2))  
  
dbsClusData = osiPCA[:, 50000, :2]  
  
eps = 0.7  
minSamples = 50  
  
dbscan = DBSCAN(  
    eps=eps,  
    min_samples=minSamples,  
    metric=manualManhattanDistance  
)  
labelsManhattan = dbscan.fit_predict(dbsClusData)  
  
uniqueLabels = np.unique(labelsManhattan)  
colors = plt.colormaps['hsv'](np.linspace(0, 1, len(uniqueLabels)))  
  
plt.figure(figsize=(10, 8))  
for i, label in enumerate(uniqueLabels):  
    clusterPoints = dbsClusData[labelsManhattan == label]  
    if label == -1:  
        plt.scatter(clusterPoints[:, 0], clusterPoints[:, 1], color='black', label='Noise')  
    else:  
        plt.scatter(clusterPoints[:, 0], clusterPoints[:, 1], color=colors[i], label=f'Cluster {label}')  
  
plt.title(f'DBSCAN Clustering (eps={eps}, min_samples={minSamples}) with Custom Manhattan Distance')  
plt.xlabel('PCA Component 1')  
plt.ylabel('PCA Component 2')  
plt.legend()  
plt.grid()  
plt.show()
```


Task 5.3: Distance functions



Cosine similarity distance function

```
def manualCosineDistance(p1, p2):
    dotProduct = sum(x1 * x2 for x1, x2 in zip(p1, p2))
    normP1 = sum(x1 ** 2 for x1 in p1) ** 0.5
    normP2 = sum(x2 ** 2 for x2 in p2) ** 0.5
    cosineSimilarity = dotProduct / (normP1 * normP2) if normP1 != 0 and normP2 != 0 else 0
    return 1 - cosineSimilarity

dbsClusData = osiPCA[:, :2]

eps = 0.05
minSamples = 5

dbscanCosine = DBSCAN(
    eps=eps,
    min_samples=minSamples,
    metric=manualCosineDistance
)
labelsCosine = dbscanCosine.fit_predict(dbsClusData)

uniqueLabels = np.unique(labelsCosine)
colors = plt.colormaps['hsv'](np.linspace(0, 1, len(uniqueLabels)))

plt.figure(figsize=(10, 8))
for i, label in enumerate(uniqueLabels):
    clusterPoints = dbsClusData[labelsCosine == label]
    if label == -1:
        plt.scatter(clusterPoints[:, 0], clusterPoints[:, 1], color='black', label='Noise')
    else:
        plt.scatter(clusterPoints[:, 0], clusterPoints[:, 1], color=colors[i], label=f'Cluster {label}')

plt.title(f'DBSCAN Clustering (eps={eps}, min_samples={minSamples}) with Custom Cosine Distance')
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
plt.legend()
plt.grid()
plt.show()
```


Task 5.4: Distance functions

```
chScoreEuclidean = calinski_harabasz_score(dbsClusData, labelsEuclidean)
chScoreManhattan = calinski_harabasz_score(dbsClusData, labelsManhattan)

print(f'Calinski-Harabasz Score for DBSCAN with Euclidean Distance: {chScoreEuclidean}')
print(f'Calinski-Harabasz Score for DBSCAN with Manhattan Distance: {chScoreManhattan}')
```

Calinski-Harabasz Score for DBSCAN with Euclidean Distance: 1629.2840206163921
Calinski-Harabasz Score for DBSCAN with Manhattan Distance: 1681.3450109572602

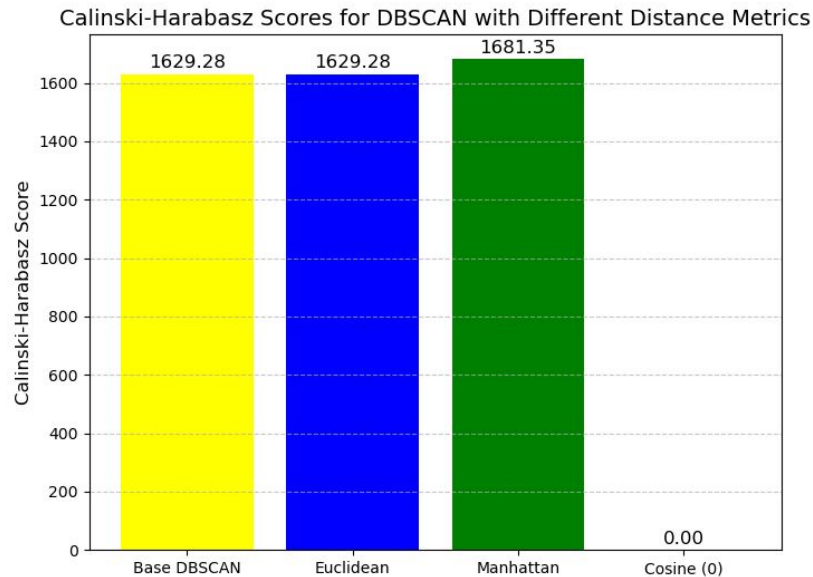
```
chScoreDBSCAN = calinski_harabasz_score(dbsClusData, dbscanLabels)
chScoreEuclidean = calinski_harabasz_score(dbsClusData, labelsEuclidean)
chScoreManhattan = calinski_harabasz_score(dbsClusData, labelsManhattan)
chScoreCosine = 0

chScores = [chScoreDBSCAN, chScoreEuclidean, chScoreManhattan, chScoreCosine]
labels = ['Base DBSCAN', 'Euclidean', 'Manhattan', 'Cosine (0)']

plt.figure(figsize=(8, 6))
plt.bar(labels, chScores, color=['yellow', 'blue', 'green', 'red'])

for i, score in enumerate(chScores):
    plt.text(i, score + 10, f'{score:.2f}', ha='center', va='bottom', fontsize=12)

plt.title('Calinski-Harabasz Scores for DBSCAN with Different Distance Metrics', fontsize=14)
plt.ylabel('Calinski-Harabasz Score', fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



Questions

